AgroSmart – Análisis de Algoritmos y Estructuras de Datos

Módulo: Clima y alertas (clima.py y pantalla_clima.py)

1. Cómo funciona el módulo

La pantalla **Clima** (`pantalla_clima.py`) toma la **localidad** del cultivo seleccionado (el departamento) y le pide los datos a `clima.py`, que los consulta a la API gratuita **Open-Meteo** (no necesita clave). El módulo usa la librería `requests`. Primero convierte el nombre de la localidad en coordenadas (servicio de geocodificación) y luego pide el clima actual o el pronóstico de varios días.

Estructura o técnica	Dónde	Para qué
Diccionario (tabla hash)	`_CACHE`	Guarda los últimos resultados por 10 minutos.
Lista de objetos `Clima`	pronóstico	Un objeto por día.
Cascada de condiciones	`generar_alerta`	Decide la alerta.
Insertion Sort y Merge Sort	`clima.py` / `ordenamiento_clima.py`	Ordenan el pronóstico por temperatura o lluvia.

2. Caché con diccionario (tabla hash)

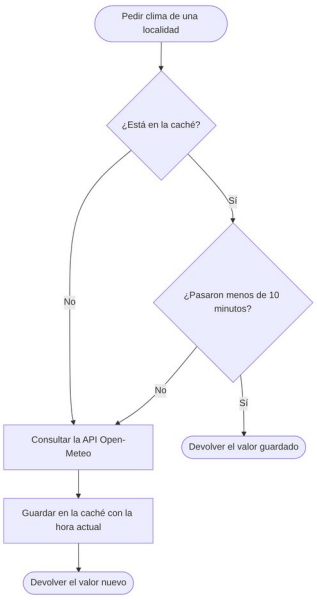
Explicación. Antes de consultar la API, se revisa si ya hay un resultado guardado para esa localidad y si pasaron menos de 10 minutos. Si es así, se devuelve de inmediato. Si no existe o ya expiró, se consulta la API, se guarda con la hora actual y se devuelve.

Pseudocódigo

```
FUNCION obtener_de_cache(clave)
    SI clave EXISTE en cache
        (valor, guardado_en) = cache[clave]
        SI (tiempo_actual - guardado_en) < 600 segundos: DEVOLVER
valor
    DEVOLVER NADA

FUNCION guardar_en_cache(clave, valor)
    cache[clave] = (valor, tiempo_actual)
```

Diagrama de flujo



Complejidad. Tiempo O(1) en promedio. Espacio O(k), con k localidades distintas consultadas.

Justificación. El servicio externo puede ser lento o tener límites de uso. La caché evita pedidos repetidos y el diccionario de Python (tabla hash) encuentra la clave de inmediato.

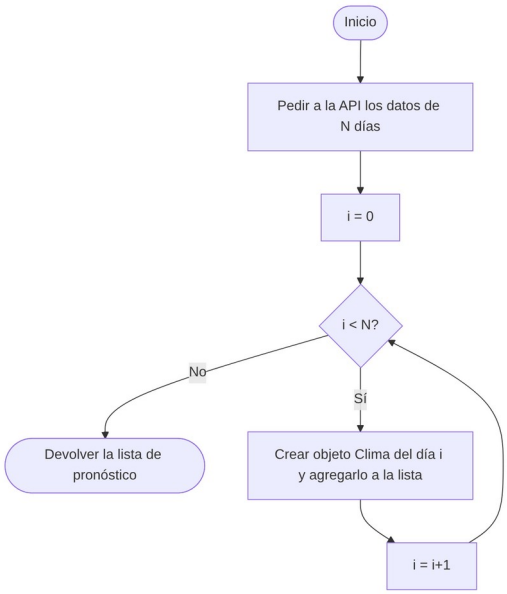
3. Recorrido lineal para el pronóstico de varios días

Explicación. La API devuelve listas con los datos de cada día (temperatura máxima, probabilidad de lluvia, código del clima). El programa las recorre de principio a fin y crea un objeto `Clima` por día. La pantalla muestra el pronóstico de **5 días**.

Pseudocódigo

```
FUNCION obtener_pronostico_dias(localidad, dias)
    datos = PEDIR_A_LA_API(localidad, dias)
    lista = []
    PARA i DESDE 0 HASTA dias - 1
        AGREGAR CREAR_CLIMA(datos[i]) A lista
    DEVOLVER lista
```

Diagrama de flujo



Complejidad. Tiempo O(n), espacio O(n), con n entre 1 y 16 días.

Justificación. Los datos de la API ya vienen en orden cronológico y son pocos, por lo que no hace falta ningún algoritmo adicional de búsqueda u ordenamiento.

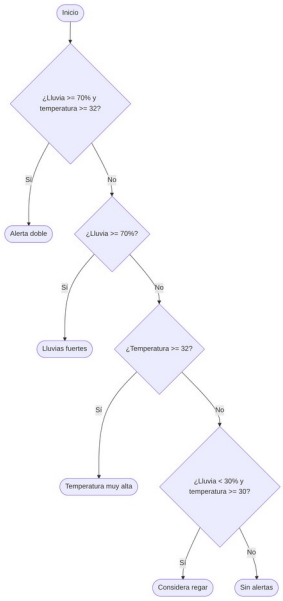
4. Árbol de decisión para las alertas

Explicación. `generar_alerta` evalúa las condiciones en orden de importancia y se detiene en la primera que se cumple. Así el agricultor recibe un solo mensaje claro.

Pseudocódigo

```
FUNCION generar_alerta(temperatura, probabilidad_lluvia)
    lluvia_fuerte = probabilidad_lluvia >= 70
    calor_extremo = temperatura >= 32
    sequia = probabilidad_lluvia < 30 Y temperatura >= 30
    SI lluvia_fuerte Y calor_extremo: DEVOLVER "Alerta doble: lluvias
fuertes y temperatura muy alta"
    SINO SI lluvia_fuerte:            DEVOLVER "Alerta: se esperan
lluvias fuertes"
    SINO SI calor_extremo:            DEVOLVER "Alerta: temperatura
muy alta"
    SINO SI sequia:                   DEVOLVER "Alerta: considera
regar tus plantas"
    SINO:                             DEVOLVER "Sin alertas por ahora"
```

Diagrama de flujo



Complejidad. Tiempo O(1) (máximo 4 comparaciones), espacio O(1).

Justificación. Una cascada ordenada de la más urgente a la menos urgente evita alertas duplicadas o confusas y no necesita estructuras adicionales.

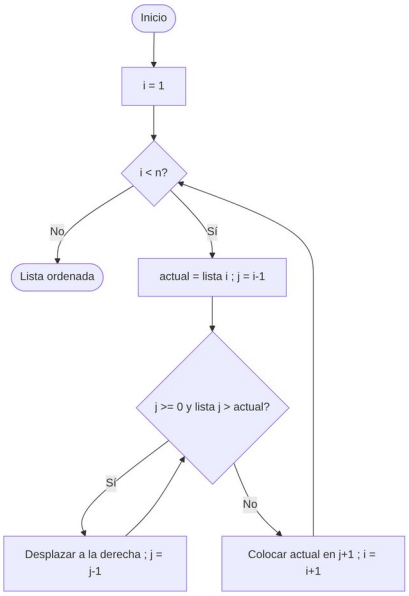
5. Ordenamiento por inserción

Explicación. Ordena el pronóstico por una clave (temperatura o probabilidad de lluvia), un elemento a la vez, insertándolo en su lugar entre los ya ordenados.

Pseudocódigo

```
FUNCION ordenamiento_insercion(lista, clave)
    PARA i DESDE 1 HASTA LONGITUD(lista) - 1
        actual = lista[i] ; j = i - 1
        MIENTRAS j >= 0 Y lista[j].clave > actual.clave
            lista[j + 1] = lista[j] ; j = j - 1
        lista[j + 1] = actual
```

Diagrama de flujo



Complejidad. Tiempo $O(n^2)$ en el peor caso y $O(n)$ en el mejor. Espacio $O(1)$.

Justificación. Para el pronóstico de 5 días (muy pocos datos), es simple, fácil de explicar y suficientemente rápida.

6. Ordenamiento por mezcla (Merge Sort)

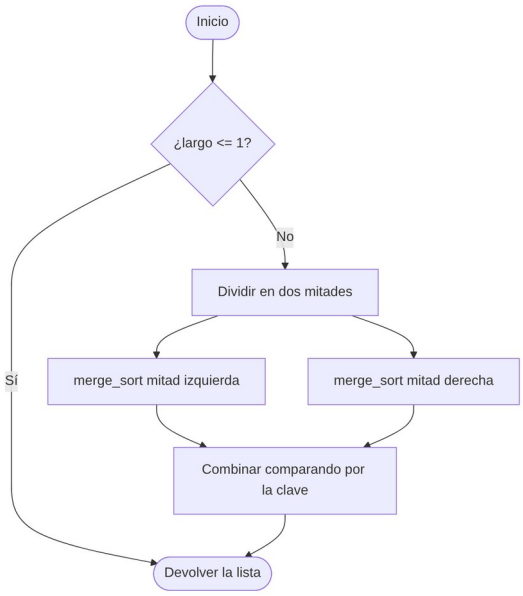
Explicación. Divide y vencerás: divide la lista a la mitad hasta tener elementos sueltos y los mezcla de dos en dos, comparando por la clave elegida. Está en `algoritmos/ordenamiento_clima.py` y `clima.py` lo usa en `generar_pronostico` para mostrar el pronóstico ordenado por temperatura y por probabilidad de lluvia.

Pseudocódigo

```
FUNCION merge_sort(lista, clave)
    SI LONGITUD(lista) <= 1: DEVOLVER lista
    medio = LONGITUD(lista) DIV 2
    izquierda = merge_sort(lista[0..medio], clave)
    derecha = merge_sort(lista[medio..fin], clave)
    DEVOLVER combinar(izquierda, derecha, clave)

FUNCION combinar(izquierda, derecha, clave)
    resultado = [] ; i = 0 ; j = 0
    MIENTRAS i < LONGITUD(izquierda) Y j < LONGITUD(derecha)
        SI izquierda[i].clave <= derecha[j].clave: AGREGAR
izquierda[i] ; i = i+1
        SINO: AGREGAR derecha[j] ; j = j+1
    AGREGAR lo que sobre de izquierda y de derecha
    DEVOLVER resultado
```

Diagrama de flujo



Complejidad. Tiempo O(n log n) siempre; espacio O(n).

Justificación. Contrasta con la inserción: usa divide y vencerás y garantiza O(n log n). Este mismo Merge Sort lo reutiliza el algoritmo voraz del reparto de agua en la pantalla Riego.

7. Manejo de errores

La pantalla valida que se haya seleccionado un cultivo y que tenga localidad. Si no hay internet, el servicio no responde o la localidad no se encuentra, se captura la excepción y se muestra el mensaje “No se pudo obtener el clima” en rojo, sin cerrar la aplicación.